

系统分析师案例题批改合集

试题一

阅读以下关于在线医疗预约系统的描述，回答问题 1 至问题 3。

【说明】

某医院拟开发在线医疗预约系统，用于管理患者预约、医生排班及诊疗记录等业务。系统主要功能如下：

患者通过系统注册账号后，可查询医生排班信息、预约门诊（含普通门诊和专家门诊）、取消预约及查看历史诊疗记录。

医生登录系统后，可维护个人排班信息、查看预约患者列表、录入诊疗记录。

管理员负责维护医生信息（新增、修改、删除）、科室信息及审核患者注册信息。

系统需对接医院 HIS 系统，定时同步患者基本信息及诊疗费用数据。

系统采用结构化分析与 UML 建模结合的方式进行设计，相关模型描述如下：

(1) 用例图中包含 "预约门诊"、"取消预约"、"查询排班" 等用例，涉及患者、医生、管理员三种角色。

(2) 数据流图中，"审核注册信息" 加工的输入数据流为 "注册申请"，输出数据流为 "审核结果"；"同步 HIS 数据" 加工与 "患者信息表" 数据存储存在双向数据流。

(3) 类图中，"患者" 类与 "预约记录" 类存在关联关系，"医生" 类与 "科室" 类存在聚合关系。

【问题 1】 (9 分)

请基于上述说明，分析用例图中以下用例之间的关系（泛化、包含、扩展），并说明理由：

- (1) "预约专家门诊" 与 "预约门诊"
- (2) "取消预约" 与 "预约门诊"
- (3) "提交预约信息" 与 "预约门诊"

【学员回答】 (9 分)

泛化关系：是一般与特殊的关系，子用例继承父用例的行为，还可添加新行为或覆盖父用例行为。

包含关系：一个用例（基础用例）必须包含另一个用例（被包含用例）的行为，被包含用例是基础用例的必要组成部分。

扩展关系：一个用例（扩展用例）对另一个用例（基础用例）的功能进行扩展，扩展用例是可选的、对基础用例的补充。

- 1、“预约专家门诊”与“预约门诊”：泛化。“预约门诊”是一般预约行为，“预约专家门诊”是其特殊形式，继承父用例行为并添加专家选择等特殊逻辑。
- 2、“取消预约”与“预约门诊”：扩展。“预约门诊”是基础用例，“取消预约”是可选的功能补充。
- 3、“提交预约信息”与“预约门诊”：包含。“预约门诊”必须包含“提交预约信息”的行为，是基础用例的必要组成部分。

【参考答案】

(1) 泛化关系。理由：“预约专家门诊”是“预约门诊”的特殊形式，继承了其基本流程并增加了专家筛选等特殊步骤，符合泛化关系中子用例对父用例的特殊化定义。

(2) 扩展关系。理由："取消预约" 是在 "预约门诊" 基础上，针对已完成预约场景的附加功能，且不影响主用例的基本流程，符合扩展关系的特征。

(3) 包含关系。理由："提交预约信息" 是 "预约门诊" 流程中必须执行的步骤，被包含用例是主用例的必要组成部分，因此属于包含关系。

【问题 2】 (8 分)

(答对一个得 4 分，答对 2 个给 8 分)

数据流图设计中存在哪些错误？请指出并说明原因。

【学员回答】 (4 分)

- 1、审核注册信息加工的输入工作流不应该是注册申请，应该是注册信息
- 2、数据流图中数据加工与数据存储之间的数据流应该是单向的，不允许双向数据流

【参考答案】

存在两处错误：

"同步 HIS 数据" 加工与 "患者信息表" 数据存储的双向数据流错误。理由：数据流图中数据存储与加工之间的数据流应明确方向，同步操作应拆分为 "读取 HIS 数据"和 "更新患者信息"两个单向数据流。

缺少 "审核注册信息" 加工与管理员角色的数据流连接。理由：加工必须有对应的外部实体触发，管理员作为审核操作的执行者，应存在 "审核指令" 的输入数据流。

【问题 3】 (8 分)

请分析类图中以下关系是否正确，并说明理由：

- (1) "患者" 类与 "预约记录" 类的关联关系及多重度 (1:N)
- (2) "医生" 类聚合 "科室" 类的关系

【学员回答】 (8 分)

- 1、患者和预约记录之间存在关联关系, 关联多重度是患者 1: 预约记录 N, 一个患者可以有
多条预约记录, 一个预约记录对应一个患者
- 2、医生类与科室类存在聚合关系, 因为两者生命周期不同, 一个医生可以在不同的科室, 医
生离职了科室也还会在, 但应该是科室类聚合了医生类

【参考答案】

- (1) 正确。理由: 一个患者可创建多条预约记录, 而一条预约记录仅属于一个患者, 符合 1:N
的多重度定义。
- (2) 不正确。理由: 科室与医生的关系应为聚合关系 (科室包含医生), 但描述中颠倒了聚
合的方向, 应是 "科室" 类聚合 "医生" 类, 而非 "医生" 类聚合 "科室" 类。

试题二

阅读以下关于系统分析与设计的叙述, 回答问题 1 至问题 3。

【说明】

某大学计划开发一套在线图书馆管理系统, 以实现图书的借阅、归还、查询、预订以及
用户信息管理等功能。系统分析师张工采用面向对象分析设计方法对该系统进行分析、
设计、建模。经过初步调研, 张工整理出以下系统需求:

- ① 图书馆管理员可以录入图书的基本信息, 包括书名、作者、ISBN 号、出版日期、
库存数量等。

- ② 用户可以以不同的方式查询图书的详细信息，并预订或借阅图书。
- ③ 用户借阅图书时，系统应记录借阅信息，包括借阅人、借阅时间、归还日期等。
- ④ 当图书库存不足时，系统应能够提醒管理员进行图书采购。
- ⑤ 用户可以撤销预订的图书。

在分析过程中，张工生成了分析模型和设计模型。

【问题 1】（10 分）

请用 200 字以内的文字阐述分析模型、分析模型建立步骤、类之间的关系。

【学员回答】（8 分）

定义：分析模型是通过类图的形式，用于从业务和需求视角，识别并描述系统中的概念类，职责，以及关联关系，完成系统的静态结构建模

步骤

- 1、定义概念类：找到系统中的对象和类
- 2、确定类之间的关系
- 3、为类添加职责
- 4、建立交互图：主要包括顺序图和协作图

关系

- 1、关联：一段时间多个类连接在一起，长期关联
- 2、聚合：整体局部关系，各自生命周期不同
- 3、组合：整体局部关系，相同生命周期

- 4、依赖：临时的，类 B 变化会引起类 A 的变化
- 5、实现：每个类实现接口的部分操作
- 6、泛化：父子类之间的继承关系

【参考答案】

分析模型：描述系统的基本逻辑结构，展示对象和类如何组成系统（静态结构），以及它们如何保持通信，实现系统行为（动态模型）。

分析模型的建立步骤：定义概念类、确认类之间的关系、为类添加职责、建立交互图。

类之间的关系：关联、聚合、组合、依赖、泛化、实现。（答出 4 点即可）

【问题 2】（6 分）

张工经过需求分析，建立了在线图书馆管理系统的状态机模型，如图 2-1 所示。请对题干需求进行仔细分析，填写图 2-1 中的（1）～（3）处空白。

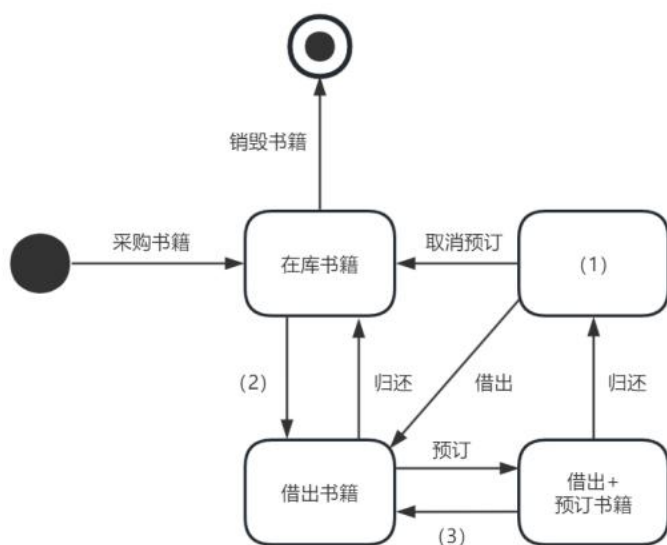


图 2-1

【学员回答】（4 分）

- 1、预定书籍
- 2、借出
- 3、取消预定

【参考答案】

- (1) 在库+预订书籍
- (2) 借出
- (3) 取消预订

【问题 3】 (9 分)

请用 200 字以内的文字说明面向对象设计中的三种类，以及每一种类的定义和作用。

【学员回答】 (6 分)

- 1、实体类：来源于分析类图的类，有一定属性，需要进行持久化衡
- 2、控制类：控制用例工作的类，体现程序的执行逻辑
- 3、边界类：用于系统接口与系统外部进行交互

【参考答案】

①实体类：

定义：映射需求中的每个实体，往往是名词，实体类有属性，但不一定有操作。

作用：用于持久化保存实体，往往对应数据库中的数据表。

②控制类：

定义：控制用例工作的类，一般用动宾结构的短语表示，控制类一般没有属性，但一定有方法。

作用：用于控制协调其他对象的工作，控制对象伴随着用例执行完毕而消亡。

③边界类：

定义：位于系统和外界的交接处，包括窗体、报表、打印机等硬件的接口，以及与其他系统的接口。

作用：每个参与者与系统至少有一个边界类，边界类使参与者能与系统交互。

试题三

阅读以下关于数据存储架构的叙述，在答题纸上回答问题 1 至问题 3。

【说明】

某电商平台为提升系统性能，采用 MySQL 作为主数据库，同时引入 Redis 作为缓存存储热点数据。系统主要支持商品信息查询、库存管理及订单存储等业务：商品详情页访问频繁，需快速响应；库存数据需实时准确，支持高并发修改；订单数据需长期保存，查询频率较低。

设计中，Redis 主要存储热门商品基本信息（如名称、价格）和实时库存数量、热销商品排名；MySQL 存储完整商品数据、库存明细、订单全量信息及用户数据。为避免数据不一致，团队还设计了 MySQL 和 Redis 的数据读写步骤。

【问题 1】（12 分）

请用 200 字以内的文字说明引入 Redis 作为缓存的核心原因。

【学员回答】（6 分）

1、提升性能：读写速度极快，可支撑高并发场景，提升系统响应效率。

- 2、数据结构丰富：支持字符串、键值对等多种结构，满足多样业务需求。
- 3、支持持久化：可持久化到磁盘，防止内存数据丢失。
- 4、扩展性强：集群模式可横向扩展节点，应对数据量与访问量增长。

【参考答案】

引入 Redis 的核心原因：

- ① Redis 基于内存存储，读写速度快，可支撑商品详情页千万级高频访问的毫秒级响应；
- ② 分担 MySQL 的读压力，避免其因高频查询过载；
- ③ 支持高并发操作，适配库存实时更新等场景；
- ④ 数据结构灵活，可高效存储热点数据（如商品信息、库存计数）。

【问题 2】（5 分）

请用 200 字以内的文字描述在商品信息更新场景中，保证 MySQL 与 Redis 数据同步的典型读写步骤。

【学员回答】（5 分）

1、读步骤：

先查询 redis，如果命中缓存，则直接返回数据

如果未命中缓存，则查询数据库，并将结果写入 redis

2、写步骤

先更新数据库，更新数据库后把数据更新到缓存

【参考答案】

典型读写步骤：

- ① 读操作：先查 Redis，命中则直接返回；未命中则查 MySQL，同时将结果写入 Redis。
- ② 写操作：先更新 MySQL 中商品信息，成功后删除 Redis 对应缓存（或直接更新 Redis），确保后续查询加载最新数据，避免缓存脏数据。

【问题 3】（8 分）

请用 200 字以内的文字说明 Redis 支持的主要数据类型；并说明如果平台需要实现“热销商品排名”功能应采用哪种数据类型及原因。

【学员回答】（6 分）

主要数据类型包括：字符串、哈希、列表、键值对等

热销产品牌名，应该用有序集合的方式进行存储

原因是有序集合内置了排序功能，可以快速进行排序

【参考答案】

Redis 主要数据类型：字符串（String）、哈希（Hash）、列表（List）、集合（Set）、有序集合（Sorted Set）。

实现“热销商品排名”应采用有序集合（Sorted Set），因它可通过“分数”（如销量）对商品排序，支持按分数范围快速查询排名，满足热销榜实时更新需求。

试题四

阅读以下说明，回答问题 1 至问题 3。

【说明】

某城市商业园区为缓解停车场管理混乱、车位利用率低、缴费排队时间长等问题，计划

开发一套智能化停车场车辆进出管理系统。该系统需实现车辆从入场到出场的全流程自动化管理，同时满足管理人员对停车场运营数据的监控与统计需求，具体需求如下：

1.车辆入场管理：车辆到达入口时，车牌识别摄像机自动捕捉车牌信息，系统查询数据库判断是否为已注册的月卡用户（企业员工、长期租户）。若为月卡用户，系统自动验证用户状态（是否在有效期内），验证通过后抬杆放行，并记录入场时间与分配的车位区域；若为临时用户，系统生成临时入场二维码（包含入场时间、入口编号），展示在入口显示屏上，用户扫码保存后抬杆放行，同时更新停车场空余车位数量。

2.车辆出场管理：月卡用户车辆到达出口时，车牌识别后直接验证用户有效性，通过后抬杆放行；临时用户需在出口缴费终端扫码（出示入场二维码），系统自动计算停车时长（出场时间-入场时间），并根据预设收费规则（如首 2 小时 10 元，后续每小时 5 元，封顶 50 元）计算费用，支持微信、支付宝扫码支付，支付成功后抬杆放行，同时记录出场时间与收费金额。

3.异常处理：当车牌识别失败（如车牌污损、遮挡）时，入口/出口岗亭的管理人员可手动输入车牌信息查询；若临时用户丢失入场二维码，管理人员可通过车牌查询入场记录，协助用户完成缴费；若系统检测到车位传感器故障（如显示“占用”但无车辆），管理人员可手动标记车位状态为“空闲”。

4.数据统计：系统每日自动生成运营报表，包含当日入场车辆总数（月卡用户/临时用户占比）、总收费金额、各时段车位使用率等数据，管理人员可登录管理后台查看报表并导出。

项目团队采用用例建模方法进行需求分析与设计，计划使用 UML 建模工具构建需求与设计模型，确保系统功能满足业务场景需求。

【问题 1】 (8 分)

(答出三个参与者和三个用例可得 8 分)

UML 用例图是需求建模的核心工具,需明确系统的参与者与用例。请根据上述需求描述,识别该停车场车辆进出管理系统的 4 个主要参与者,并列举 4 个核心用例,同时简要说明每个用例的功能。

【学员回答】 (4 分)

参与者: 车辆用户、摄像机, 岗亭管理员, 车位传感器, 系统时钟

用例:

- 1、用户注册为月卡用户。功能: 是否是月卡用户, 用来判断车辆进入时是直接抬杆还是需要先扫码再抬杆, 用来判断车辆离开时是否进行收费,
- 2、摄像头识别车牌, 包含车辆进入时识别, 车辆离开时识别。功能: 识别到车牌才能触发是否抬杆驶入或者是否收费。
- 3、车辆入场记录。功能: 入场记录决定了零时用户的收费价格, 以及异常发生时管理员手动收费。
- 4、系统每日自动生成报表。功能: 满足管理员对停车场数据每日的监控和统计需求

【参考答案】

主要参与者如下:

- 1.车主: 包括注册用户(已开通账号)和临时用户, 核心操作是车辆入场、出场缴费、查询车位;
- 2.停车场管理员: 负责处理异常出场(如无入场记录、欠费争议)、维护车位状态、查看运营数据;

3.财务人员：通过系统获取收费报表，核对账目；

4.系统设备：车牌识别设备（自动识别车牌）、车位传感器（采集车位状态）、引导屏（展示车位信息）。

主要用例如下：

1.车辆入场登记：功能是通过车牌识别设备获取车牌信息，区分注册/临时用户，记录入场时间，同步车位状态至监控模块；

2.车辆出场缴费：功能是识别车牌并查询入场记录，计算停车费用，支持自动扣款（注册用户）、现金/移动支付（临时用户），缴费成功后触发道闸开启；

3.车位状态监控：功能是通过传感器实时采集车位状态（空闲/占用/故障），在管理端和引导屏展示车位分布，更新空余车位数；

4.收费报表生成：功能是按指定周期（日/周/月）统计停车收入、车辆流量，生成报表供财务人员核对、管理人员分析；

5.异常出场处理：功能是当车辆出场时无入场记录、欠费争议等异常，系统提示管理员介入，记录处理结果并允许手动放行；

6.收费规则配置：功能是支持管理员新增/修改收费规则（如时长费率、分时段价格、特殊车辆优惠），规则生效后自动同步至收费计算模块。

【问题 2】（12 分）

UML 中的顺序图与交互图（协作图）均用于描述对象间的动态交互，但适用场景存在差异。请回答以下两个问题：

（1）简要说明顺序图与交互图（协作图）的核心区别；

(2) 列出顺序图包含的 4 种核心元素，并简要说明每种元素的作用。

【学员回答】 (6 分)

答案一

顺序图的核心是强调用例的执行顺序，是注重时间。交互图的核心是强调用例之间的协作，注重交互。

答案二

生命线：代表交互中的参与者，通常是一个对象实例

激活条：表示一个正在执行的一个操作或者消息时段，展示了对象的活动时间

消息：对象之间通信的载体

交互片段：描述复杂逻辑的交互

【参考答案】

顺序图与交互图（协作图）的核心区别：

1.顺序图：

侧重点：以“时间顺序”为核心，清晰展示对象间消息传递的先后顺序，强调“何时交互”；

适用场景：需梳理复杂业务流程的时间逻辑，或向团队 / 客户直观演示步骤先后（如用户操作流程、系统模块间的时序交互）。

2.协作图：

侧重点：以“对象关联”为核心，展示对象间的连接关系，强调“谁与谁交互”，消息传递顺序通过序号标识；

适用场景：需分析对象间的依赖关系，或优化系统模块结构（如梳理类间协作关系、排查模块耦合问题）。

顺序图包含的 4 种核心元素，并简要说明每种元素的作用：

1. 参与者 (Actor)

指与系统发生交互的外部实体，既可以是用户（如停车场的“临时用户”“管理人员”），也可以是外部硬件/系统（如“车牌识别设备”）。其核心作用是作为消息的发起或接收源头，明确系统交互的外部触发主体，界定系统与外部环境的边界。例如在“临时用户出场缴费”流程中，“临时用户”作为参与者发起“扫码上传入场二维码”的消息，是流程的起始触发者。

2. 对象 (Object)

指系统内部承载具体业务逻辑的功能模块或实体，通常对应面向对象设计中的“类”的实例，在顺序图中以“对象名:类名”的格式表示（如“FeeCalc:收费计算类”“BarrierCtrl:道闸控制类”）。其作用是执行核心业务操作，是消息传递的核心载体。例如“收费计算类 (FeeCalc)”接收“临时用户扫码后的数据”，执行“计算停车费用”的逻辑，是实现缴费功能的关键对象。

3. 生命线 (Lifeline)

是从参与者或对象底部延伸的纵向虚线，代表参与者/对象在时间维度上的存在周期。其作用是通过纵向时间轴体现交互的先后顺序——消息需沿生命线纵向排列，上方消息先发生，下方消息后发生，直观展示“何时交互”。例如在“临时用户出场缴费”顺序图中，“临时用户”的生命线起始处发送“扫码”消息，后续“收费计算类”的生命线接收消息并返回“费用结果”，生命线的纵向位置清晰反映了流程的时序。

4. 消息 (Message)

指参与者与对象、对象与对象之间传递的交互指令，是顺序图中描述“交互内容”的核心元素，通过带箭头的线条表示，箭头上方需标注消息名称（如“临时用户→缴费终端：扫码上传入场二维码”“收费计算类→支付模块：请求验证支付状态”）。其作用是明确交互的具体内容与

流向，串联起整个业务流程的逻辑节点。例如“道闸控制类”接收“出场管理类”发送的“开启道闸”消息后，执行抬杆操作，消息成为触发硬件动作的关键指令。

【问题 3】 (5 分)

请简要说明构建用例模型包含了哪个步骤。

【学员回答】 (5 分)

- 1、识别参与者
- 2、合并需求得到用例
- 3、细化用例
- 4、调整用例

Ps：前三个步骤是必然的，第四个根据具体情况判断是否参与

【参考答案】

1. 识别参与者
2. 合并需求获得用例
3. 细化用例描述
4. 调整用例模型

试题五

阅读以下说明，回答问题 1 至问题 3。

【说明】

某市市民中心为提升公共文化服务体验，避免灯光秀观赏时出现人员拥挤、秩序混乱等

问题，计划开发一套灯光秀预约管理系统。该系统需实现市民预约、预约审核、现场核验、数据统计等功能，具体需求如下：

1. 市民预约：市民通过系统注册账号（支持手机号、身份证号注册）后，可选择预约日期（未来 7 天内开放预约）、场次（每日 3 场：19:00-19:30、20:00-20:30、21:00-21:30）及预约人数（单场最大预约人数 500 人），提交预约申请时需填写同行人员信息（姓名、身份证号），系统自动校验预约人数是否超出单场限额。
2. 预约审核：管理员登录系统后，可查看待审核预约申请，审核依据包括 “市民是否存在历史爽约记录（30 天内爽约 2 次及以上则拒绝预约）” “同行人员信息是否真实（对接公安身份验证接口）”，审核通过后系统向市民发送预约成功短信（含预约码），审核拒绝则反馈拒绝原因。
3. 现场核验：市民到达现场后，工作人员通过核验终端扫描市民预约码，系统验证预约码有效性（是否在对应场次有效期内、是否已核验），验证通过后记录核验状态，同时更新现场已入场人数；若预约码无效（如过期、已核验），系统提示工作人员拒绝入场。
4. 数据统计：系统每日自动生成预约统计报表，包含 “各场次预约量、审核通过率、现场核验率、爽约率” 等数据，管理员可查看报表并导出（支持 PDF、Excel 格式），用于优化灯光秀场次安排与预约规则。

项目团队采用面向对象方法进行系统分析与设计，计划通过 UML 建模工具构建分析模型与设计模型，确保系统功能满足业务需求。

【问题 1】（10 分）

面向对象分析中，建立系统分析模型是核心环节。请用不超过 200 字阐述建立市民中心灯光秀预约管理系统分析模型的主要步骤。

【学员回答】 (9 分)

- 1、首先定义概念类，为类添加职责，建立交互图
 - 阅读用例描述，找出名词或者名词短语建立初始清单类
 - 把类分为显而易见的类，不确定的类，毫无意义 的类
 - 舍弃掉毫无意义的类，把不确定的类通过小组讨论再分成显而易见的类和毫无意义的类
 - 重复上述步骤
- 2、确定类之间的关系，类之间的关系有关联，组合，聚合，泛化，实现，依赖，找出关系后使用 uml 记录形成领域模型
- 3、根据类的属性或者功能为每个类添加职责
- 4、最后使用 UML 建立交互图

【参考答案】

1. 定义概念类
2. 确定类之间的关系
3. 为类添加职责
4. 建立交互图

前三个步骤统称为类 责任 协作者(Class Responsibility Collaborator,CRC)建模”，意味着“定义概念类”“确定类之间的关系”“为类添加职责”这三步聚焦于明确类本身是什么（类）、要做什么（责任）以及和其他类如何配合（协作者，即类之间的关系），是构建分析模型中关于类的基础且关键的部分，后续“建立交互图”则侧重于展现类实例的动态交互过程。

【问题 2】 (8 分)

类与类之间存在多种关联关系。请列出 UML 中类的 4 种常见关系, 并结合市民中心灯光秀预约管理系统的类设计, 为每种关系分别举例说明 (需明确涉及的类及关系含义)。

【学员回答】 (6 分)

关联关系, 组合关系, 聚合关系, 泛化关系, 实现关系, 依赖关系

聚合关系: 整体和部分的关系, 具有不同的生命周期。这里预约场次和市民是聚合关系, 因为场次里面里面包含了预约的市民, 但是市民是可以脱离于场次而独立存在。

依赖关系: 一个类依赖于另外一个类, 类的变化可能引起另外一个类的变化, 比如同行人和预约市民是依赖关系, 同行人的依赖于同行市民, 同行人的变化可能导致市民的变化。

【参考答案】**1. 关联关系**

关系定义: 描述类与类之间稳定的双向或单向交互关系, 体现“谁与谁有业务关联”, 分为一对一、一对多、多对多;

系统示例: “市民类 (Citizen)” 与 “预约记录类 (ReservationRecord)” 为一对多关联关系——1 个市民可提交多个预约记录 (如不同日期的场次预约), 1 个预约记录仅属于 1 个市民, 关联的业务含义是“预约记录需明确归属的市民, 以便追溯预约来源”。

2. 依赖关系

关系定义: 描述类 A (依赖方) 需借助类 B (被依赖方) 的功能才能完成自身操作, 是临时性、单向的弱关联, 若 B 变更会影响 A;

系统示例: “预约审核类 (AuditManager)” 依赖 “公安身份验证接口类 (IDVerifyAPI)”——审核类的 verifyCompanion() 方法 (验证同行人员身份) 需调用接口类的 verifyIDCard()

方法，若接口类的方法参数或返回格式变更，审核类的对应方法需同步调整，符合依赖关系的“单向依赖、变更影响”特性。

3. 聚合关系

关系定义：描述“整体与部分”的关系，部分可脱离整体独立存在，体现“包含但非依赖”的逻辑；

系统示例：“灯光秀场次类 (ShowSession)”与“预约记录类 (ReservationRecord)”为聚合关系——场次是“整体”，预约记录是“部分”，1 个场次包含多个预约记录，但预约记录可脱离场次存在（如预约被拒绝后，记录仍保存在系统中用于统计），符合“部分可独立”的聚合特性。

4. 继承关系

关系定义：描述“一般类（父类）与特殊类（子类）”的关系，子类继承父类的属性与方法，同时可扩展自身特有属性/方法，体现“共性与个性”；

系统示例：定义“用户类 (User, 父类)”，属性包括“用户 ID、账号、密码、登录状态”，方法包括“login()、logout()”；“市民类 (Citizen)”与“管理员类 (Admin)”作为子类继承用户类——市民类扩展“爽约次数”属性与“submitReservation()”方法，管理员类扩展“审核权限等级”属性与“auditReservation()”方法，既复用父类的登录/登出功能，又体现各自的特殊业务属性。

【问题 3】 (7 分)

请用 200 字以内的文字说明面向对象设计中的三种类，以及每一种类的定义和作用。

【学员回答】 (7 分)

实体类

定义：用例的实体，一般由系统的参与者组成

作用：数据的载体，对应着数据库表结构

控制类

定义：用例执行的某个动作或功能，封装了业务逻辑

作用：一般用来控制实体类进行数据的操作

边界类

定义：位于系统和外界的交接处，是系统与外部参与者交互的中介

作用：接收外部数据把数据传输到系统内部，输出数据到外部，比如屏幕或外部接口

【参考答案】

①实体类：

定义：映射需求中的每个实体，往往是名词，实体类有属性，但不一定有操作。

作用：用于持久化保存实体，往往对应数据库中的数据表。

②控制类：

定义：控制用例工作的类，一般用动宾结构的短语表示，控制类一般没有属性，但一定有方法。

作用：用于控制协调其他对象的工作，控制对象伴随着用例执行完毕而消亡。

③边界类：

定义：位于系统和外界的交接处，包括窗体、报表、打印机等硬件的接口，以及与其他系统的接口。

作用：每个参与者与系统至少有一个边界类，边界类使参与者能与系统交互。

试题六

阅读以下关于 Web 系统架构的设计的叙述，回答下列问题。

【说明】

某电商公司拟开发一个可以支持亿级流量的系统。公司架构师张工认为应该采用 B/S 技术实现系统开发。王工认为采用当前主流的微服务架构可以更好的支持高并发的场景应用。公司经过讨论分析最终采用了王工的建议。

其系统的架构如下图所示的设计。

客户端	移动设备和 PC 端
访问层	(1)、上游 <u>nginx</u> 和应用 <u>nginx</u>
接口层	<u>WebSocket</u> 、(2)、TCP/UDP、API 网关
服务层	服务通信：(3) 和 MQ 。 服务的组间实现商品服务、订单服务、营销服务等。
	任务调度：(4) 和 Elastic Job 。基础服务组件有设备管理、隔离管理、缓存管理等。
存储接入层	<u>MyCat</u> 、(5) 和 Redis Proxy
存储层	NOSQL 包括 (6) 和 <u>Solr</u> 、Elastic Search

【问题 1】 (12 分)

分析该系统架构，从下列选项中选择对应的数据填入 (1) - (6) 中。

LVSA B.HTTP/HTTPS C.DUBBO D.Quartz E.MongDB F.ES Client

【学员回答】 (6 分)

(1) F ; (2) B ; (3) C ; (4) A ; (5) C ; (6) E。

【参考答案】

(1) A ; (2) B ; (3) C ; (4) D ; (5) F ; (6) E。

【问题 2】 (5 分)

什么是微服务？微服务架构有哪些优缺点？

【学员回答】 (4 分)

某个或多个单一的功能组成服务叫做微服务，是比分布式服务更小的服务单元。

优点：

微服务单独部署简单，单独部署时不用考虑其他服务的部署功能

微服务具有自治性和独立性，一个服务出问题了不会影响到其他服务

可拓展性高，可根据资源部署随时增加或减少微服务数量

缺点：

总体部署复杂，需要考虑到多点的部署以及通讯

数据同步需要经过接口传输，可能出现数据不一致的问题

【参考答案】

微服务架构建议将大型复杂的单体架构应用划分为一组微小的服务，每个微服务根据其负责的具体业务职责提炼为单一的业务功能；每个服务可以很容易地部署并发布到生产环境里隔离和独立的进程内部，它可以很容易地扩展和变更；对于一个具体的服务来说可以采用任何适用的语言和工具来快速实现；服务之间基于基础设施互相协同工作。

微服务的优势：

- (1) 解决了复杂性问题。它把庞大的单一模块应用分解为一系列的服务，同时保持总体功能不变。
- (2) 让每个服务能够独立开发，开发者能够自由选择可行的技术，让服务来决定 API 约定。
- (3) 每个微服务都能独立配置，开发者不必协调对于本地服务配置上的变化，这种变化一旦

测试完成就被配置了。

(4) 让每个服务都可以独立调整，你可以给每个服务配置正好满足容量和可用性限制的实例数。

微服务架构带来的挑战：

- (1) 并非所有的系统都能转成微服务。例如一些数据库层的底层操作是不推荐服务化的。
- (2) 部署较以往架构更加复杂：系统由众多微服务搭建，每个微服务需要单独部署，从而增加部署的复杂度，容器技术能够解决这一问题。
- (3) 性能问题：由于微服务注重独立性，互相通信时只能通过标准接口，可能产生延迟或调用出错。例如一个服务需要访问另一个服务的数据，只能通过服务间接口来进行数据传输，如果是频繁访问，则可能带来较大的延迟。
- (4) 数据一致性问题：作为分布式部署的微服务，在保持数据一致性方面需要比传统架构更加困难。

【问题 3】 (8 分)

本案例在数据存储层采用的是 NOSQL 数据库，请指出 NOSQL 数据库有哪些优缺点。

【学员回答】 (5 分)

优点

- 1、灵活的数据模型，支持多种数据格式
- 2、拓展性好
- 3、存储量大，高性能
- 4、高可用，能对应各种故障

缺点

1、关联性弱，连表查询不友好

【参考答案】

NoSQL 数据库的优点：

- (1) 支持高并发数据访问，性能较高。
- (2) NoSQL 数据库的数据存储结构松散，能够灵活支持多种类型的数据格式。
- (3) NoSQL 数据库能够支持海量数据的存储，且易于横向扩展。
- (4) NoSQL 数据库基于分布式数据存储，不存在单点故障和性能瓶颈，系统可用性高。

NoSQL 数据库的缺点，该系统采用 NoSQL 数据库时可能存在的问题有：

- (1) NoSQL 数据库的现有产品不够成熟，大多数产品处于初创期。
- (2) NoSQL 数据库并未形成一定的标准，产品种类繁多，缺乏官方支持。
- (3) NoSQL 数据库不提供对 SQL 的支持，学习和应用迁移成本较高。

找到我们 Find us



51CTO 软考网站
rk.51cto.com



51CTO 软考题库
网页/小程序



软考交流群